

实验细节

根据实验材料中的线索和细节，首先我锁定了**银行家算法**，然而该算法是用于避免死锁的，而非检测死锁的。接下来我找到《操作系统概念(原书第 10 版)》，根据书中一个类似于银行家算法的死锁检测法。

在用户态完成实现并进行测试后，我将其移植到内核态，并由每一个进程进行维护，同时将 `add_thread`、`add_resource`、`request_resources`、`release_resources` 等函数交由相关系统调用进行使用，成功通过了测试样例。

问答题

问题一

答：根据 `exit_current_and_run_next` 函数中的代码可以看出来，需要回收如下的资源：

1. 回收所有线程的 `TaskControlBlockInner`，也就是说，当前进程下的所有的线程都不会作为一个执行流运行下去了
2. 回收进程的 `pid`，并标记为僵尸进程，等待父进程进行回收。
3. 回收子进程，交给 `init` 进程进行管理。
4. 回收进程的物理内存空间。

其他线程可能会在调度器中被引用，也可能处于阻塞状态，在 `timer` 中被引用，都是需要回收的。因为进程是线程资源的容器，进程被回收，所有线程都应该被回收。

问题二

答：`Mutex1` 和 `Mutex2` 的区别是 `Mutex1` 使用了 `loop`，这样当使用它的线程被调度时，会执行一次 `mutex_inner.locked = true`；进行加锁。另一方面 `Mutex2` 在中进行 `unlock` 时不修改 `mutex_inner.locked` 的值，直接把控制权交给下一个线程。

可以看出来 `Mutex1` 并不能保证队列中的下一个线程能够拿到锁，因为 `mutex_inner.locked = false`；后，就意味着有线程能够进入临界区了，如果此时有线程被调度进去，那么拿到的锁的一定不是队列中的下一个线程。

荣誉准则

1. 在完成本次实验的过程（含此前学习的过程）中，我曾分别与 以下各位 就（与本次实验相关的）以下方面做过交流，还在代码中对应的位置以注释形式记录了具体的交流对象及内容：
 - 我们伟大的群友们
2. 此外，我也参考了 以下资料，还在代码中对应的位置以注释形式记录了具体的参考来源及内容：

- 《RISC-V 开放架构设计之道 1.0.0 (原著 The RISC-V Reader: An Open Architecture Atlas)》
 - 《RISC-V Supervisor Binary Interface Specification》
 - KiMi, Deepseek
 - 《Linux 内核深度解析》
 - 《The RISC-V Instruction Set Manual: Volume II》
 - 《图解 Linux 内核 基于 6.x》
 - 《操作系统概念(原书第 10 版)》
3. 我独立完成了本次实验除以上方面之外的所有工作，包括代码与文档。我清楚地知道，从以上方面获得的信息在一定程度上降低了实验难度，可能会影响起评分。
 4. 我从未使用过他人的代码，不管是原封不动地复制，还是经过了某些等价转换。我未曾也不会向他人（含此后各届同学）复制或公开我的实验代码，我有义务妥善保管好它们。我提交至本实验的评测系统的代码，均无意于破坏或妨碍任何计算机系统的正常运转。我清楚地知道，以上情况均为本课程纪律所禁止，若违反，对应的实验成绩将按“-100”分计。